

Deep Learning

Session 7: Natural language processing (1)

October 15, 2024

This session will be recorded for in-class use.

Logistics

Course Midterm Feedback

Thank you for providing feedback!

- 12 respondents largely felt:
 - Content of the class useful and interesting
 - Pace and style of lectures mostly fine, for some too fast
 - Lab pace too high
 - Some struggle with coding, most want more fundamentals of coding in the lab
 - Some want more assignments and practice, others less
- Please fill out if you haven't done so
- Additional technical lecture (5.11.) or more review (29.10.)?

Deep learning theory

1. Deep learning in public policy
2. Deep neural networks (1)
3. Deep neural networks (2)

Applications

4. Computer vision (1): Convolutional neural networks
5. Computer vision (2): Modern CNNs, implementation, and applications
6. Sequence methods and time-series analysis
7. **Natural language processing (1)**
8. Natural language processing (2): Transformer models, implementation, and applications
- Midterm exam -
9. Further topics in deep learning

Deep learning and public policy

10. Policy approaches to regulate deep learning and responsible implementation in government
11. Deep learning in practice
12. Tutorial presentations

Natural language processing (1)

- Text and public policy
- Word embeddings
- Word2Vec with focus on skip-gram model
- Continuous bag of words, GloVe
- Sentiment analysis
- Regularization

Introduction

Text and public policy

Text and public policy

- Language is a main form of communication
- Political and legislative work is formulated in text
- Political texts include laws, transcribed parliamentary speeches, party manifestos, etc.
- Social media generates a flood of texts that can be political (e.g. Twitter X data)
- Political science and public policy research has long tradition of manually “coding” and analysing text
- “Text as data” approaches are emerging
- Deep learning is a powerful tool for analyzing text automatically at scale



EUROPEAN COMMISSION

Brussels, 21.4.2021

COM(2021) 206 final

2021/0106(COD)

Proposal for a

REGULATION OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL

LAYING DOWN HARMONISED RULES ON ARTIFICIAL INTELLIGENCE (ARTIFICIAL INTELLIGENCE ACT) AND AMENDING CERTAIN UNION LEGISLATIVE ACTS

{SEC(2021) 167 final} - {SWD(2021) 84 final} - {SWD(2021) 85 final}

EXPLANATORY MEMORANDUM

1. CONTEXT OF THE PROPOSAL

1.1. Reasons for and objectives of the proposal

This explanatory memorandum accompanies the proposal for a Regulation laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Artificial Intelligence (AI) is a fast evolving family of technologies that can bring a wide array of economic and societal benefits across the entire spectrum of industries and social activities. By improving prediction, optimising operations and resource allocation, and personalising service delivery, the use of artificial intelligence can support socially and environmentally beneficial outcomes and provide key competitive advantages to companies and the European economy. Such action is especially needed in high-impact sectors, including climate change, environment and health, the public sector, finance, mobility, home affairs and agriculture. However, the same elements and techniques that power the socio-economic benefits of AI can also bring about new risks or negative consequences for individuals or the society. In light of the speed of technological change and possible challenges, the EU is committed to strive for a balanced approach. It is in the Union interest to preserve the

Text and public policy

Analyzing party manifestos

Bilbao-Jayo and Almeida (2018)

- Manifesto Project: annotated parties' election manifestos
- 56 categories grouped into seven major policy areas: external relations, freedom and democracy, political system, economy, welfare and quality of life and social groups.
- Sentence classification with convolutional neural networks in seven different languages: Spanish, Finnish, Danish, English, German, French, and Italian

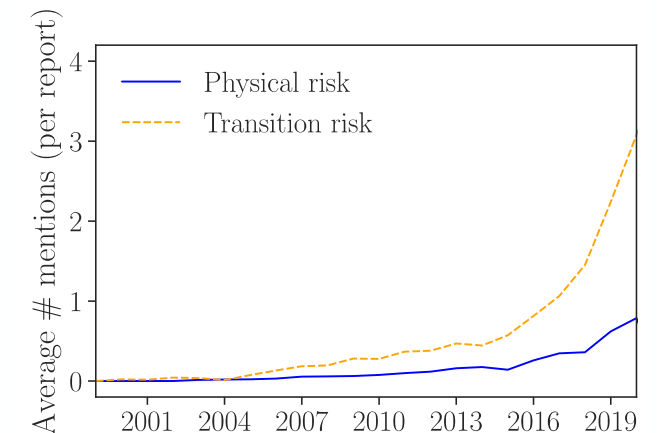


equity issue: most languages are not natively represented by ML models

Identifying climate risk disclosure

Friedrich et al. (2021)

- Corpus of >5000 corporate annual reports
- Annotated paragraphs for climate-related disclosure of financial risks
- Relevant for investment decisions and ex-post policy evaluation
- Document classification with transformer model BERT



Analysing text

Example NLP tasks common in deep learning

- Text classification:
 - Sentence classification
 - Document classification
 - Sentiment analysis
 - Natural language inference matching text to other text statements
- Question answering
- Named entity recognition
- Part-of-speech tagging
- Text summarization
- Machine translation
- Text completion
- Text-to-image and image-to-text generation
- Dialogue management
- ...

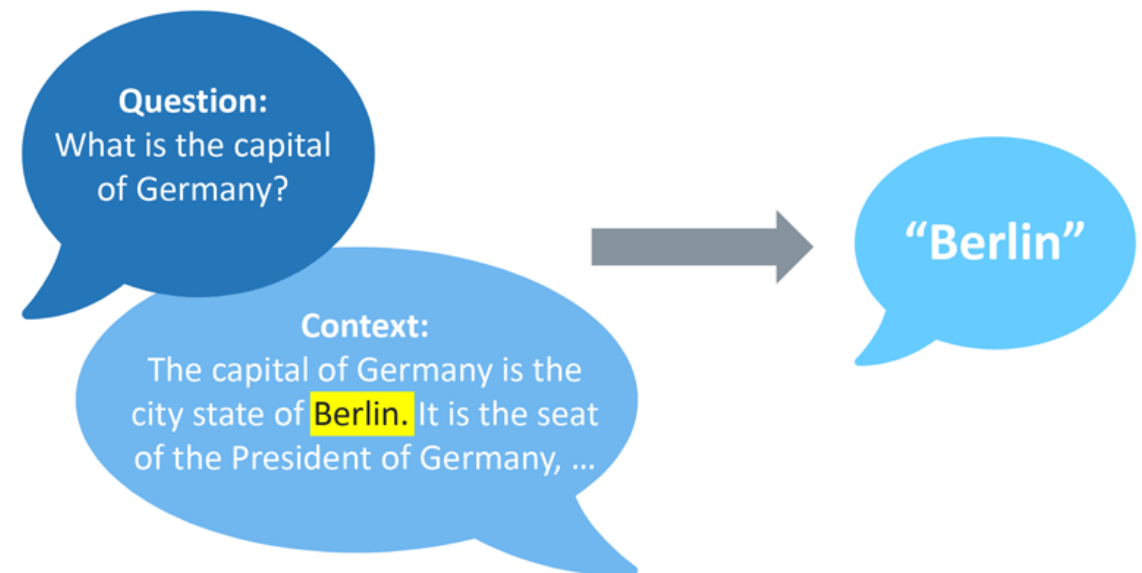
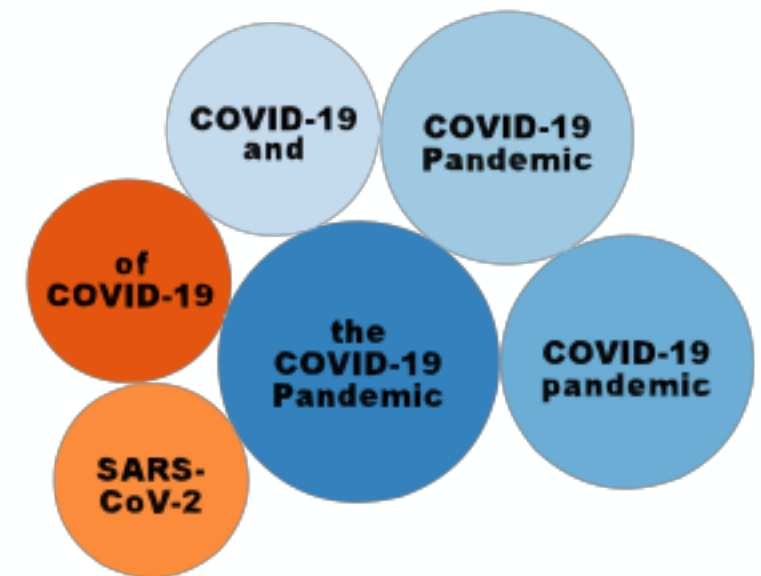


Image: Seeburger

Analyzing text

Text as data

- Text is a **string** of characters
- The string can be broken into **tokens**: words or characters, numbers, and punctuation marks.
- Text is a **sequence**: tokens follow another to make sense.
- A **corpus** is a collection of **documents**, which are each texts of various lengths.
- A **vocabulary** is the collection of tokens in the corpus.
- An **n-gram** combines n of these tokens into a short sequence.
- The process of representing the text as numerical data is called **embedding**.

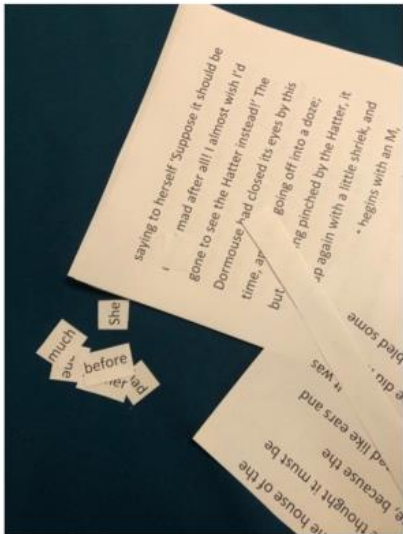


n-grams

Analysing text

Document embedding: Bag of words

- Ignore word order
- Count the number of times a word appears



1. the red dog →

2. cat eats dog →

3. dog eats food →

4. red cat eats →

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

basic vectorisation of documents - but it doesn't preserve word order

Analysing text

an extension of bag of words

Document embedding: Tf-idf (term frequency inverse document frequency)

- Gives a weight to each word in each document
- For each document, count occurrences of each word
- Compare the frequency of each word in a document with the frequency of that word occurring over the whole corpus

Word embedding

- Vector space, where each word is assigned a vector
- Neural network trained to output these vectors
- Vectors that are close corresponds to words that appear in similar context
- One can do computation with vectors

$$\textit{king} - \textit{man} + \textit{woman} =$$

Document embedding



Analysing text

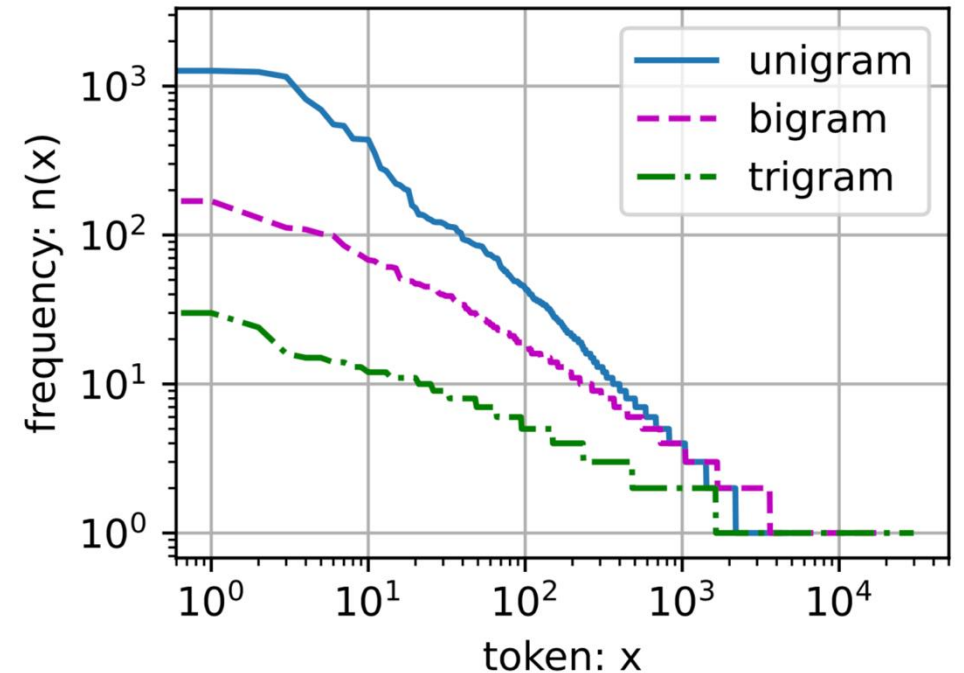
Preprocessing

1. Load text as strings into memory.
 2. **Tokenization:** Split the strings into tokens (e.g., words or characters). when using pre trained:
need to match the tokenizer to the model
 3. Build a vocabulary dictionary to associate each vocabulary element with a numerical index.
 4. Convert the text into sequences of numerical indices.
- Tokenization can mean splitting into words, word-pieces, characters, etc. depending on problem.
 - Tokenizers are readily available for NLP applications.
 - When building the vocabulary, rare elements are often dropped and represented by a special token in the text (e.g. , "<unk>" – unknown value)

Analysing text

Preprocessing

- Lowercasing
- Stop-word removal: Removing frequent words that do not add meaning (the, and, are, ...)
- Stemming: Reduce every word from its inflected form to its stem – e.g. {develop, developing, development} -> develop
- Lemmatization: Reduce every word from its inflected form to its lemma (dictionary form) – e.g. {drive, drove, driven} -> drive; {easily} -> easili



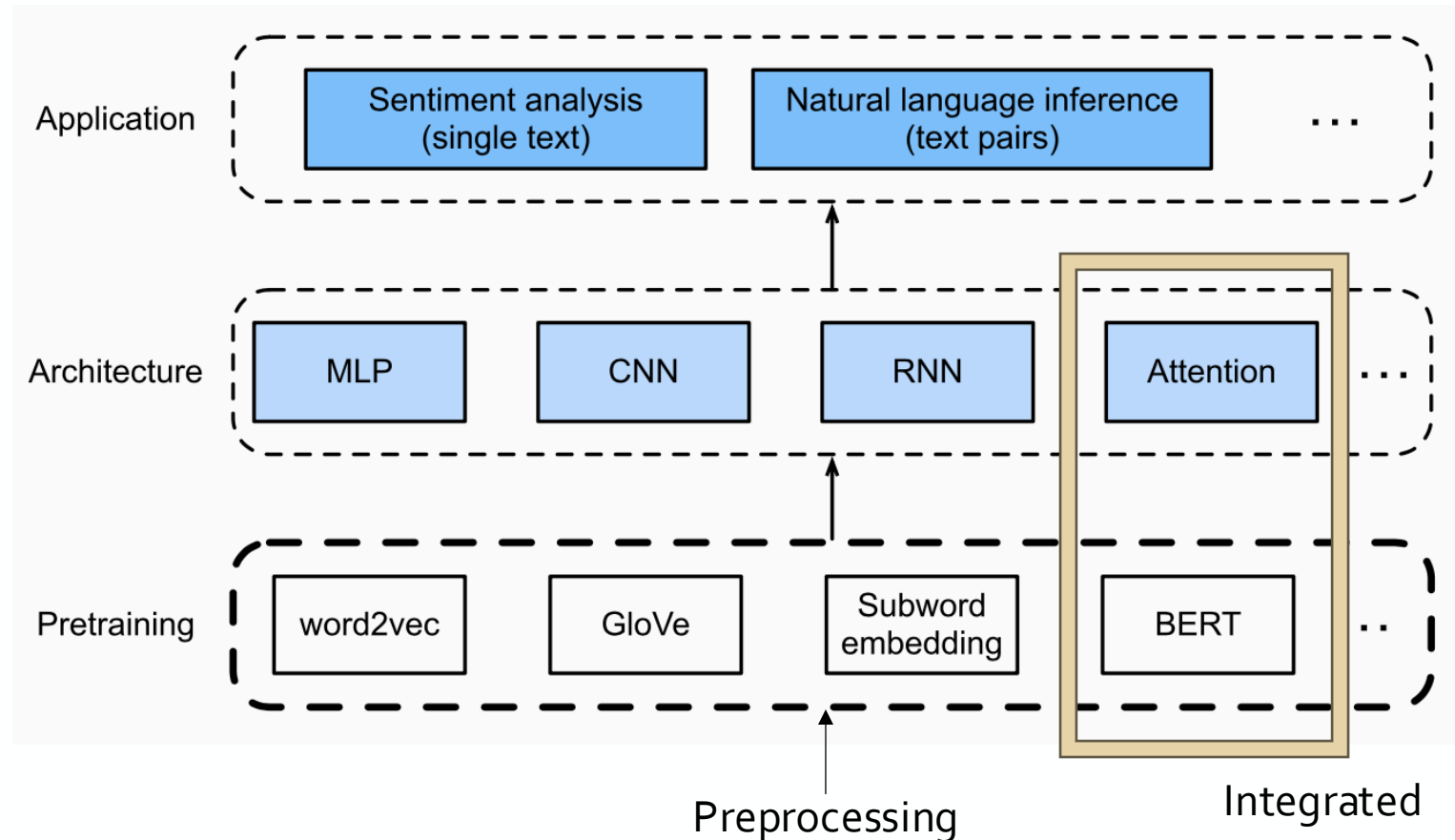
Analysing text

Simple approach to document classification

1. Tokenize and preprocess the text as desired
 2. Apply the bag of words model
 3. Apply tf-idf weighting to improve accuracy
 4. Use the resulting features in a classifier (e.g., support vector machine, gradient boosting, random forest, ...)
- Works often perfectly fine for simple tasks
 - Small model (less parameters to train)
 - To improve accuracy: learned embeddings, better classifiers (e.g., LSTM for short texts), deep-learning based contextual embeddings and classifiers (transformers)

LSTMs maintain the sequence structure

Analysing text



General structure of NLP using deep learning

- Some modularity in the process
- Word embedding $\leftrightarrow$ feature extraction
- Word embeddings are at the basis of all models (sometimes integrated with the model)

Word embeddings

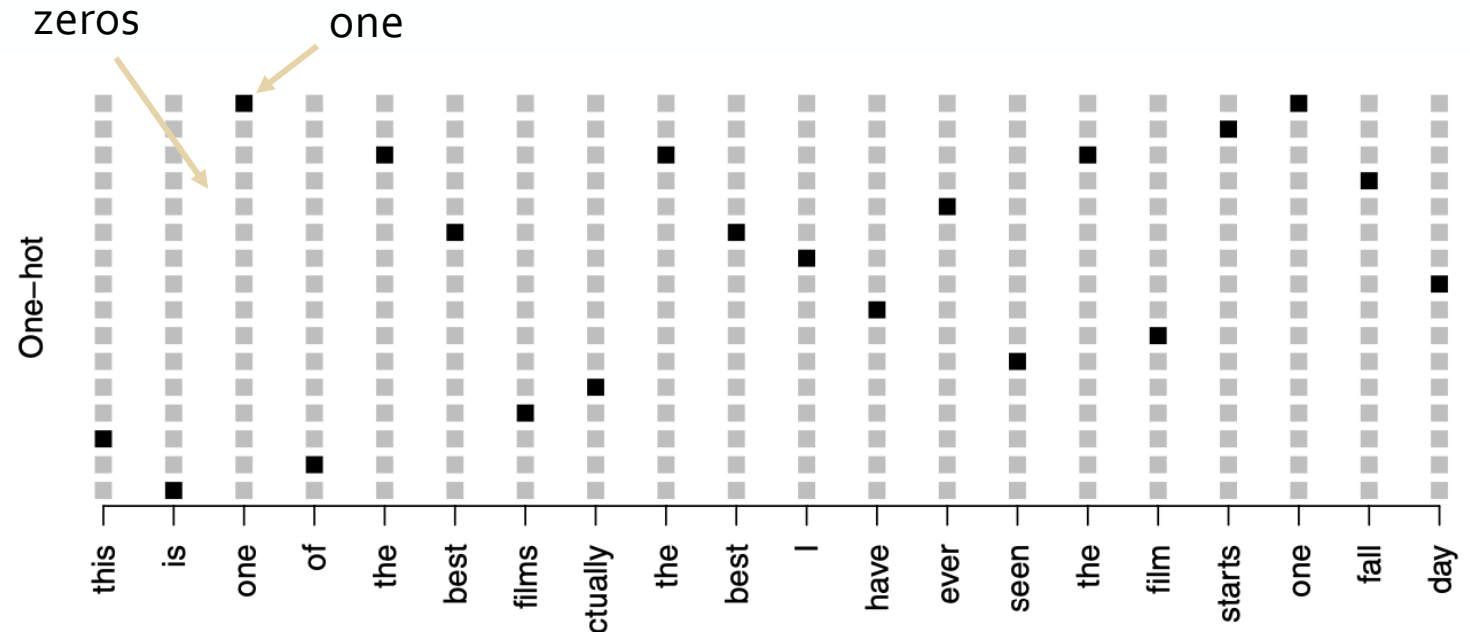
Zhang et al. and other sources

Embedding

One-hot encoding

“this is one of the best films actually the best I have ever seen the film starts one fall day”

- Simple representation: One-hot encoding of word/token
- Length of vector is size of dictionary of a fixed set of words
- Problem: Similarity between different words does not matter



problem: each of these vectors are orthogonal to each other --> we don't get a sense of relationship between them --> we use cosine-similarity with embeddings

SEE UPDATED LECTURE SLIDES THERE'S A BUNCH OF USEFUL STUFF HERE IN THIS PLACE

ADD COSINE SIMILARITY

+

"We need a continuous vector representation for embeddings"

Word2Vec

Continuous word embeddings

- Word2Vec published and patented by Mikolov et al. at Google in 2013
- Cosine similarity indicates the level of semantic similarity between two words
- Group of models based on shallow neural networks that learn embeddings
- Vectors typically have several hundred dimensions
- Trained with large datasets in an unsupervised setting
- Widely used in NLP
- Extended with doc2vec and other models such as BioVectors for biological sequences
- Two methods: skip-gram and continuous bag-of-words (CBOW)

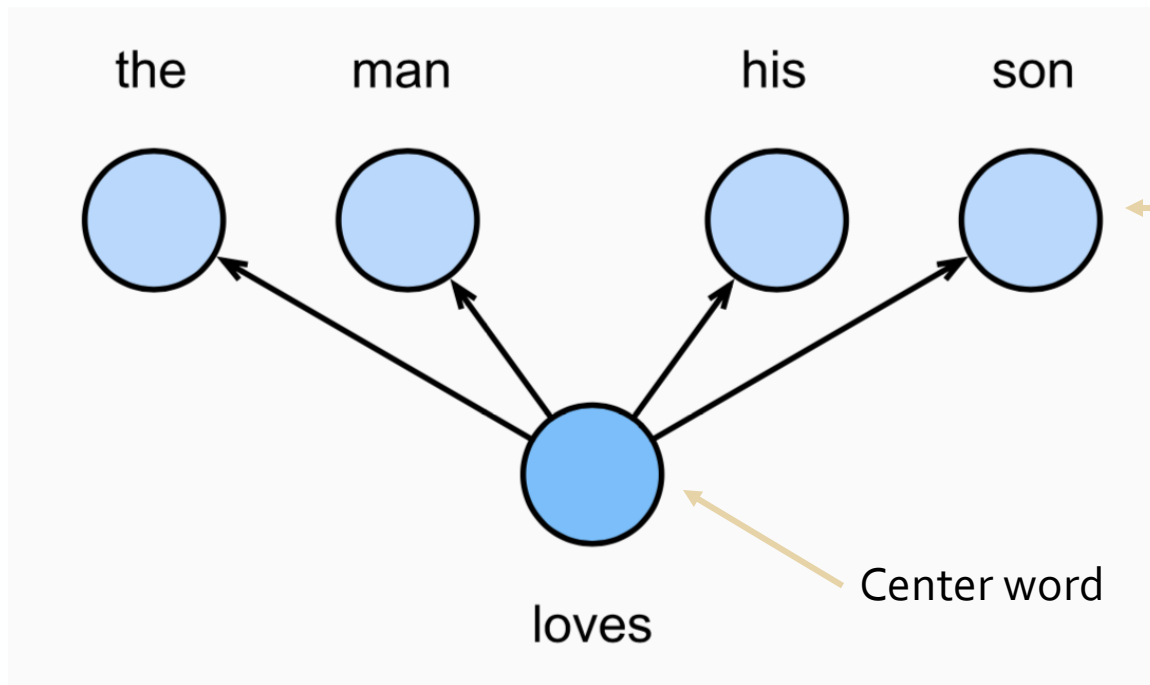
$$\textit{king} - \textit{man} + \textit{woman} = \textit{queen}$$

doc2vec: used for document retrieval / or for getting specific pages within a large document
can look for cosine similarity

Word2Vec

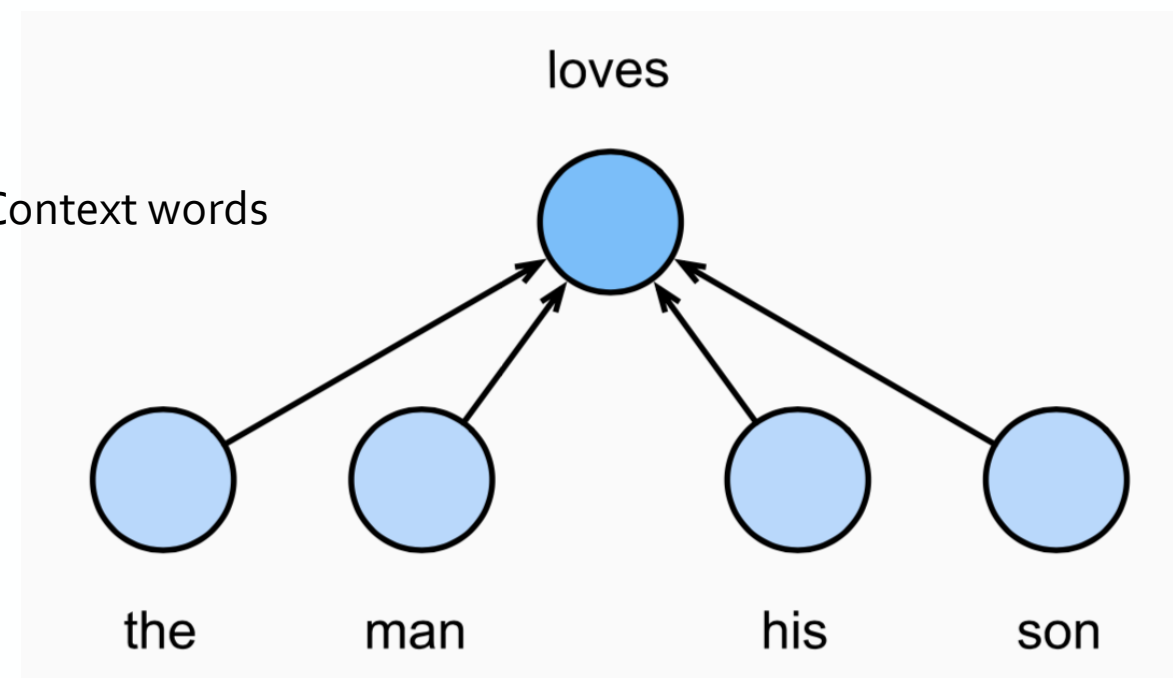
Skip-Gram Model

$$P(\text{"the", "man", "his", "son"} | \text{"loves"})$$



Continuous Bag of Words (CBOW) Model

$$P(\text{"loves"} | \text{"the", "man", "his", "son"})$$



Word2Vec

Skip-Gram Model

Assumption: context words are **independently** generated given the center word:

$$P(\text{"the", "man", "his", "son"} | \text{"loves"}) \\ = P(\text{"the"} | \text{"loves"}) \cdot P(\text{"man"} | \text{"loves"}) \cdot P(\text{"his"} | \text{"loves"}) \cdot P(\text{"son"} | \text{"loves"})$$

we assume: conditionally independent given "loves" = a basic assumption behind this model

Every word is expressed with two d -dimensional vectors to compute these conditional probabilities:
For any word in the dictionary with index i , the vector of it when used as a **center word** is $\mathbf{v}_i \in \mathbb{R}^d$,
and when used as a **context word** it is $\mathbf{u}_i \in \mathbb{R}^d$.

Using a softmax operation, we can express the conditional probability of context word w_o given center word w_c as

$$P(w_o | w_c) = \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^T \mathbf{v}_c)}$$

nb with the embeddings, we can either choose the $\mathbf{u}$ or the $\mathbf{v}$

I DON'T UNDERSTAND: IF THE GROUND TRUTH LABEL IS THE 1-HOT ENCODED LABEL WHAT IS THE ROLE OF THE PROB FUNCTION???

with vocabulary index set $\mathcal{V} = \{0, 1, \dots, |\mathcal{V}| - 1\}$.

WHERE DOES THE PROBABILISTIC MODEL COME ON IN,
IF WE ARE THROWING IN INPUT AND OUTPUT AND THE NN LEARNS THE REST?

Word2Vec

Skip-Gram Model

NB each word appears in both u and v - each word has 2 expressions depending if its context or centre

We now find the optimal embedding $\mathbf{u}_i$ and $\mathbf{v}_i$ for all words in the vocabulary by maximizing the likelihood.

For a sequence of length T with words $w^{(t)}$ at step t and context window of size m (in the running example we have $m = 2$), the likelihood is

$$\mathcal{L}(\text{parameters}) = \prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w^{(t+j)} | w^{(t)}).$$

The loss function based on the log-likelihood is

$$-\sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w^{(t+j)} | w^{(t)}).$$

check this is in latex

Word2Vec

Skip-Gram Model - Example

For a sequence of length T with words $w^{(t)}$ at step t and context window of size m the likelihood is

$$\prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w^{(t+j)} | w^{(t)})$$

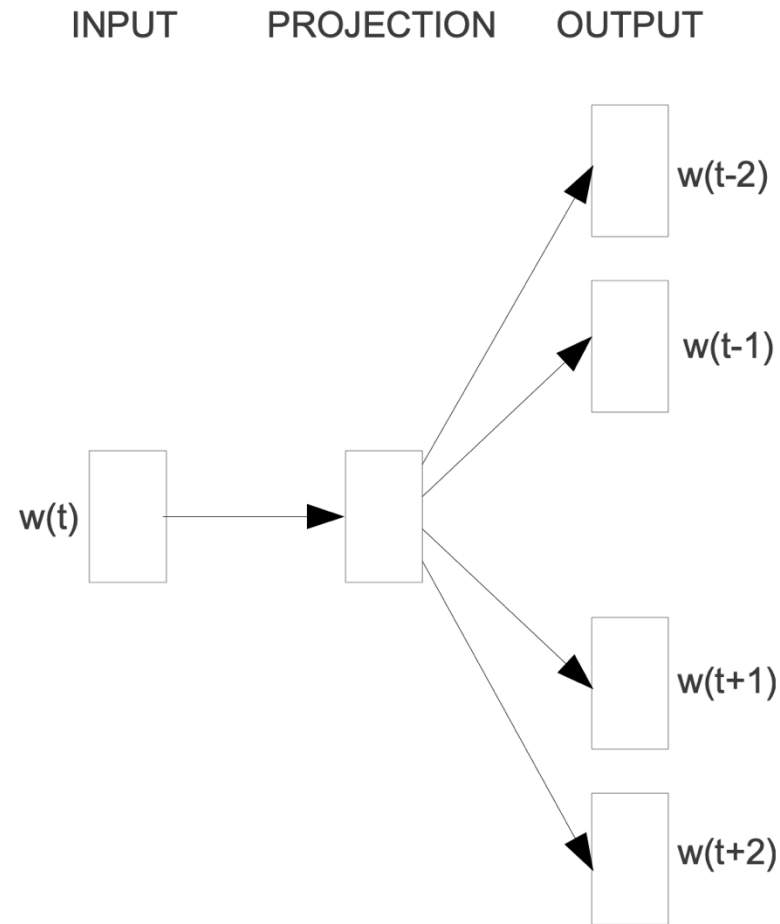
Example: “the man loves his son” ($m = 2$), $w^{(t)} = \text{“loves”}$

$$\prod_{-m \leq j \leq m, j \neq 0} P(w^{(t+j)} | w^{(t)})$$

$$= P(\text{“the”} | \text{“loves”}) \cdot P(\text{“man”} | \text{“loves”}) \cdot P(\text{“his”} | \text{“loves”}) \cdot P(\text{“son”} | \text{“loves”})$$

We would then take the product over all center words in the sequence, not only $w^{(t)} = \text{“loves”}$.

Word2Vec



Skip-gram

Skip-Gram Model

Prediction task:

Predict the context words $w^{(t+j)}$ based on the center

word $w^{(t)}$ it's a prediction task where we don't care about the actual prediction, but rather the embedded weights

We ultimately don't care about the prediction, we want to extract the learned parameters!

The model is trained on word pairs (of center and context word pairs).

Word2Vec

Training the Skip-Gram Model

- Training data: Word-pairs if they co-occur as center and context word within the context window (example on the right)
- Build the vocabulary $\mathcal{V}$ and one-hot-encode each word
- Input x : One-hot encoded center word (of dimension $|\mathcal{V}|$)
- Output $\hat{y}$: Vector of size $|\mathcal{V}|$ of continuous values
- Ground truth y : One-hot encoded context word (of dimension $|\mathcal{V}|$)
- Word embeddings are learned weights

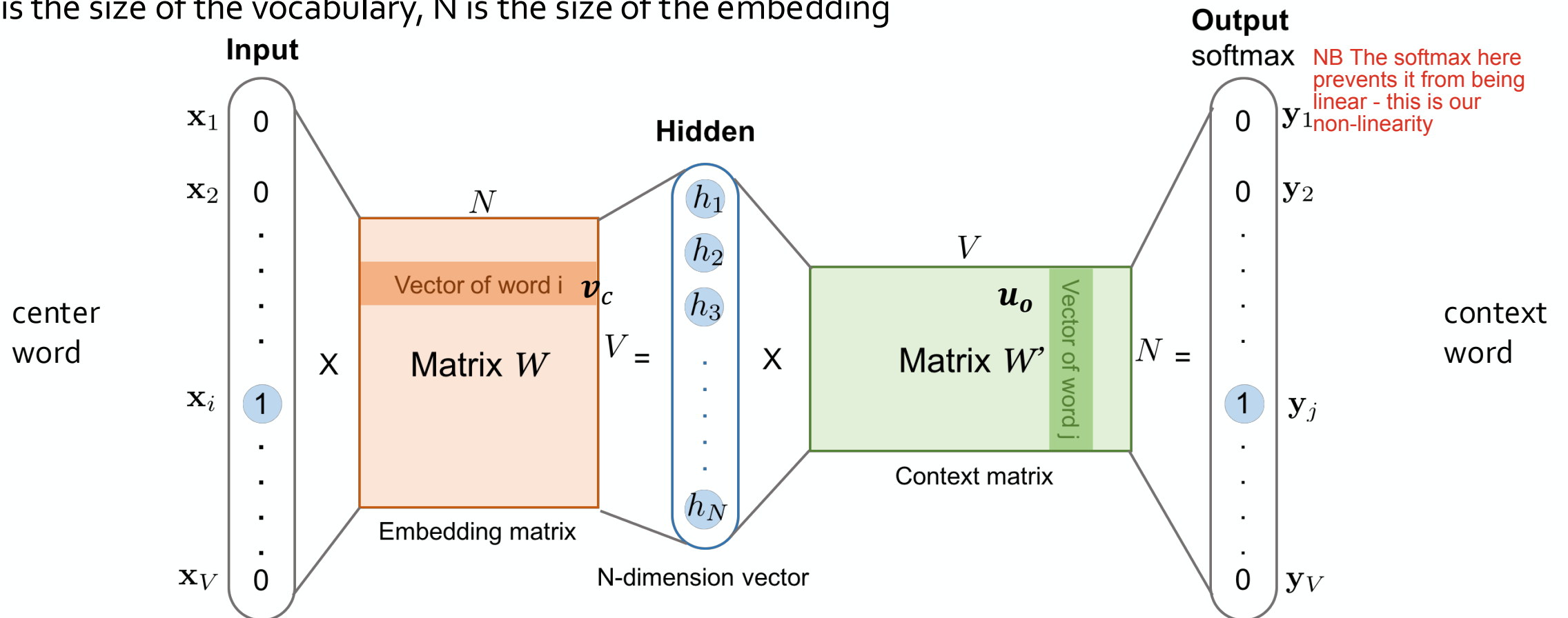
Source Text	Training Samples						
<table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡	The	quick	brown	(the, quick) (the, brown)			
The	quick	brown					
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡	The	quick	brown	fox	(quick, the) (quick, brown) (quick, fox)		
The	quick	brown	fox				
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡	The	quick	brown	fox	jumps	(brown, the) (brown, quick) (brown, fox) (brown, jumps)	
The	quick	brown	fox	jumps			
<table><tr><td>The</td><td>quick</td><td>brown</td><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡	The	quick	brown	fox	jumps	over	(fox, quick) (fox, brown) (fox, jumps) (fox, over)
The	quick	brown	fox	jumps	over		

Word2Vec

in Skip-gram we are taking the centre word here (v_c) to be the vector
we could also use the context word in the second matrix (u_o) but it's not customary for SkipGram

Training the Skip-Gram Model

V is the size of the vocabulary, N is the size of the embedding



Word2Vec

Training the Skip-Gram Model

Using the log-likelihood as a loss function, for every word pair:

$$\log P(w_o|w_c) = \log \frac{\exp(\mathbf{u}_o^T \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^T \mathbf{v}_c)} \stackrel{\text{RHS: simplified expression after some log rules have been applied}}{=} \mathbf{u}_o^T \mathbf{v}_c - \log \left(\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^T \mathbf{v}_c) \right)$$

now we want to take deriv wrt to weights we want to learn (essentially this is back propagation)

we can compute a gradient update with respect to the parameters (e.g. the center word vector $\mathbf{v}_c$ by **taking the derivative of the loss**

$$\begin{aligned} \frac{\partial \log P(w_o|w_c)}{\partial \mathbf{v}_c} &= \mathbf{u}_o - \frac{1}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^T \mathbf{v}_c)} \sum_{j \in \mathcal{V}} \exp(\mathbf{u}_j^T \mathbf{v}_c) \mathbf{u}_j \\ &= \mathbf{u}_o - \sum_{j \in \mathcal{V}} \left(\frac{\exp(\mathbf{u}_j^T \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^T \mathbf{v}_c)} \right) \mathbf{u}_j = \mathbf{u}_o - \sum_{j \in \mathcal{V}} P(w_j|w_c) \mathbf{u}_j \end{aligned}$$

Problem: $\mathcal{V}$ is the vocab - we need the conditional probs of all the words in the vocab conditional on the centre word (???) this is HUGE
computationally challenging
.... Trick is to add some noise through negative sampling (next)

Word embeddings are typically the center word vectors

Word2Vec

Lowering computational costs: Negative sampling

- Positive datapoint: ([“brown”, “quick”], 1)
- Negative (noise) datapoint: ([“fox”, “dog”], 0)

Model the probability of the words co-occurring as center and context word within the context window using a sigmoid

$$P(D = 1 | w_c, w_o) = \sigma(\mathbf{u}_o^T \mathbf{v}_c)$$

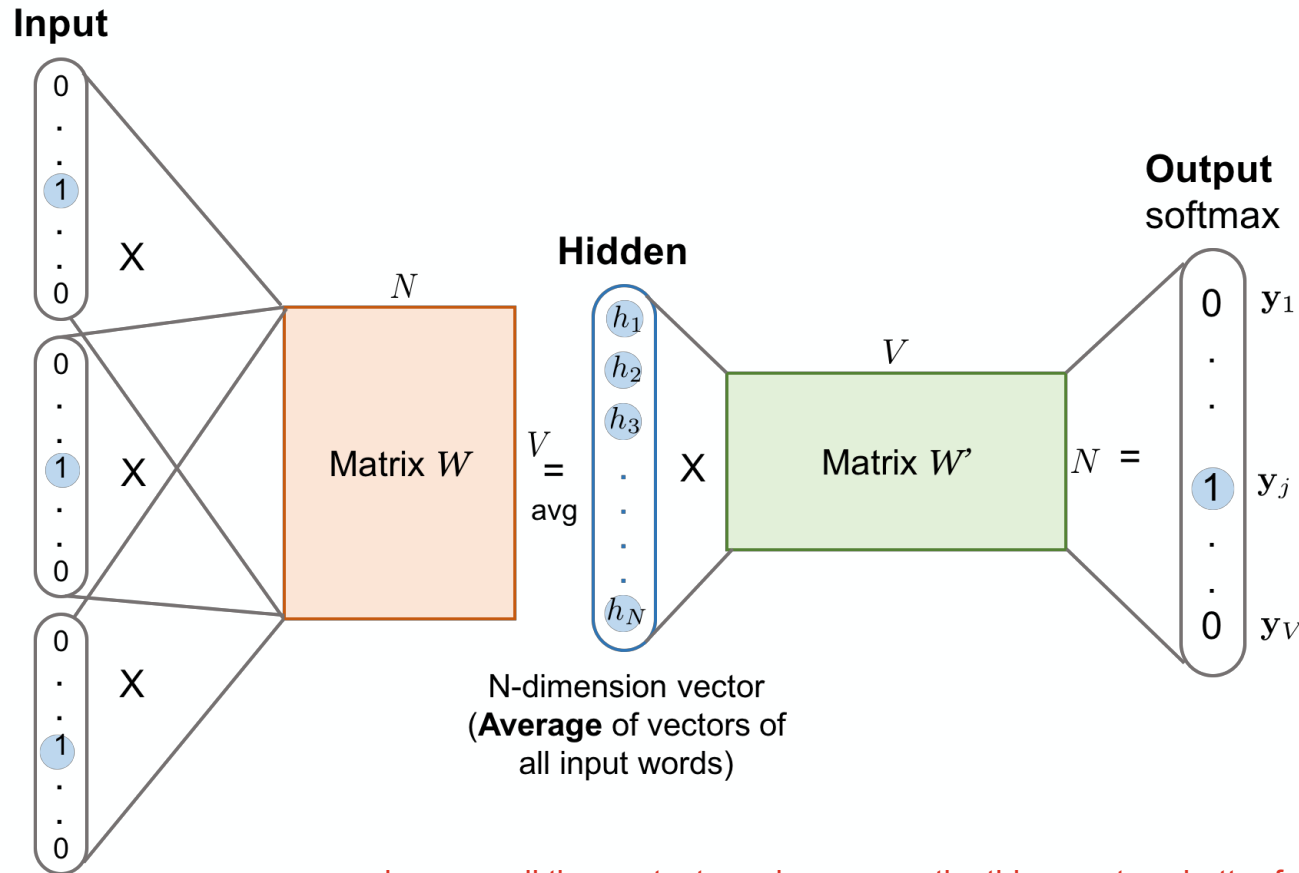
nb here we modify the original expr - incl wrap it in a sigmoid to model these conditional probabilities

That allows to approximate the likelihood function (and therefore loss function) using the set of K negative examples.

The computational costs for gradients, now depend linearly on the number of noise examples K .

Source Text	Training Samples
The quick brown fox jumps over the lazy dog.	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog.	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog.	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog.	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

Word2Vec



Continuous Bag of Words (CBOW) Model

- Center word predicted from context words
 $P(\text{"loves"} | \text{"the", "man", "his", "son"})$
- Word embeddings are typically the context word vectors
- May be better for small data because of averaging over the context words and reducing overfitting.

averaging over all the context words --> smooths things out --> better for small datasets

NB in CBOW we are using the context word vectors for the embeddings --> Lynn thought this was in W' , but I think it's actually W

GloVe

Word Embedding with Global Vectors (GloVe)

- Modification of the skip-gram model
- Not asymmetric conditional probabilities (i.e. center and context), but symmetric co-occurrences of words
- The center word vector and the context word vector are mathematically equivalent for any word in GloVe.
- GloVe uses the squared loss to fit precomputed global corpus statistics.

Contextual word embeddings

Intuition

- In the word embedding methods covered just now, the word “bank” would be represented by the same vector:
 - “I have money in the **bank**.”
 - “**Bank** fishing a river is a great way to catch a lot of smallmouth bass.”
- Many words have several meanings dependent on the context.
- Modern NLP approaches relying on deep learning use contextual embeddings.
- Depending on the context, a token is embedded differently.
- Much better representation of language
- → Next session



Sentiment analysis

Zhang et al.

Sentiment analysis

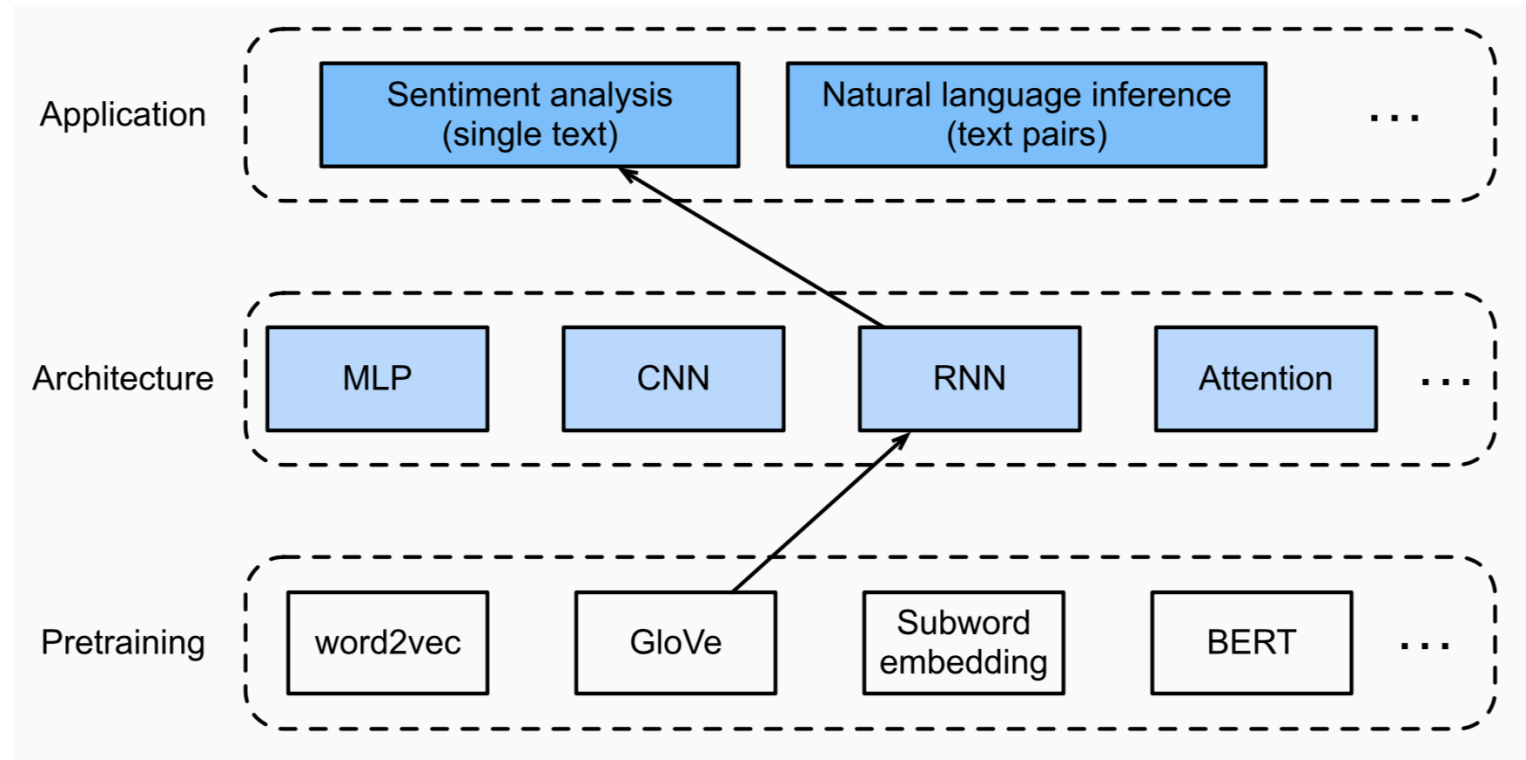
- Short documents are classified into “negative” and “positive” sentiment
- Other scales and classes are possible, including “neutral” sentiment

"The more I use @salesforce the more I dislike it. It's slow and full of bugs. There are elements of the UI that look like they haven't been updated since 2006. Current frustration: app exchange pages won't stop refreshing every 10 seconds" --> "Negative".

"That's what I love about @salesforce. That it's about relationships and about caring about people and it's not only about business and money. Thanks for caring about #TrailblazerCommunity" --> "Positive".

"Coming Home: #Dreamforce Returns to San Francisco for 20th Anniversary. Learn more: <http://bit.ly/3AgwOOH> via @Salesforce" --> "Neutral" as it doesn't contain an opinion or polarity.

Sentiment analysis



Sentiment analysis approach

We combine a pretraining approach with a deep learning architecture to perform the classification task.

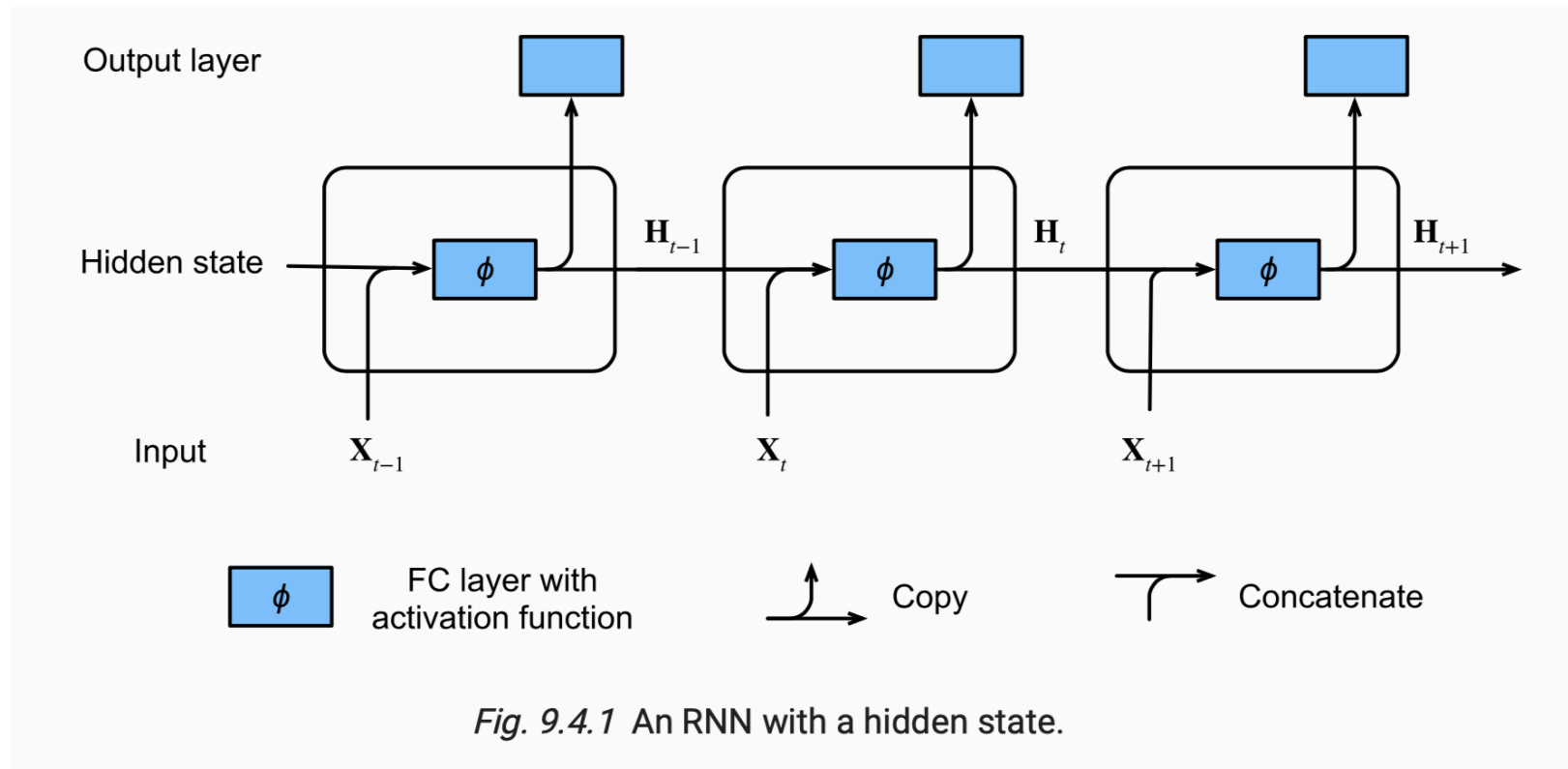
RNN: Varying length text sequence will be transformed into fixed categories

Sentiment analysis with RNN

Recurrent neural network (recap)

Basic principle of an RNN

More advanced covered last session (e.g., LSTM)



Sentiment analysis with RNN

RNN for language modeling

Character sequence "machine" [m, a, c, h, i, n, e]

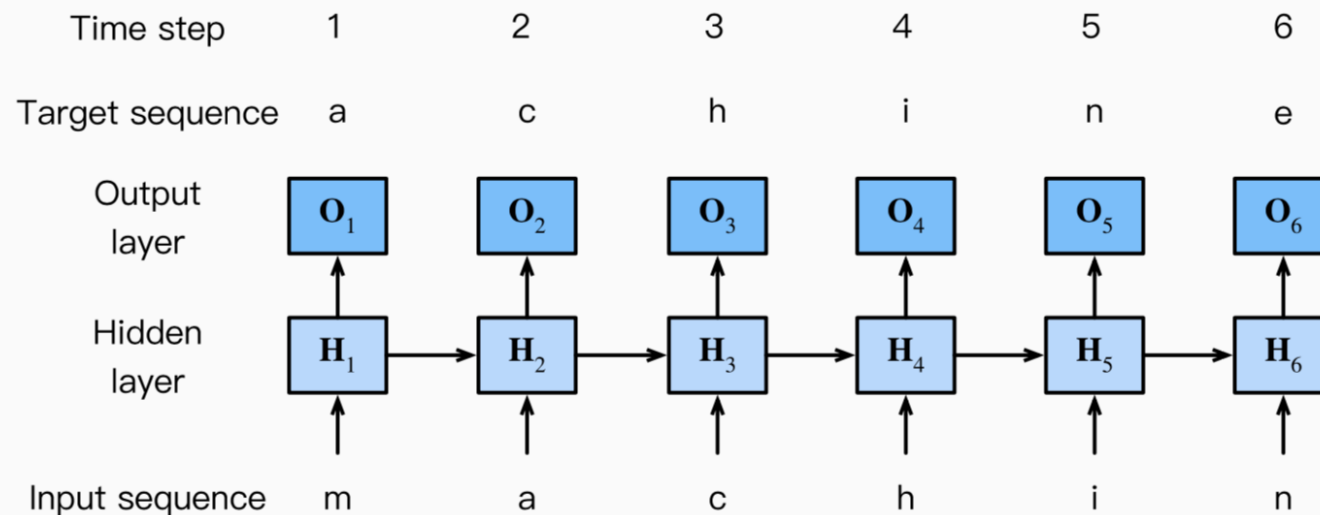


Fig. 9.4.2 A character-level language model based on the RNN. The input and target sequences are "machin" and "achine", respectively.

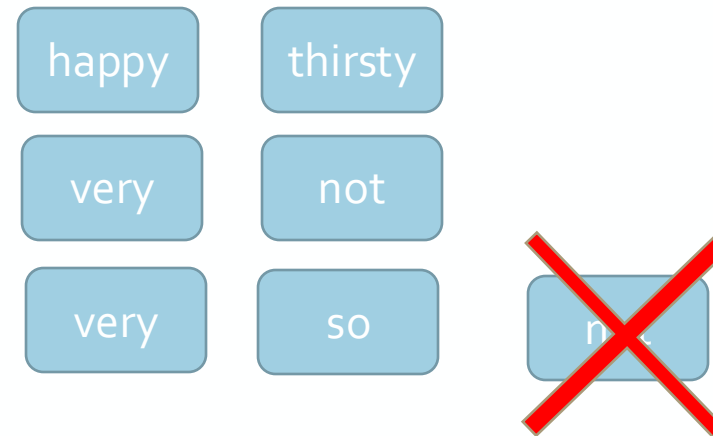
Sentiment analysis with RNN

Bidirectional RNNs

Pretraining task: masking out tokens and then to train a sequence model to predict the values of the missing tokens.

Depending on what comes after the blank, the likely value of the missing token changes:

- I am _____. *can be basically anything*
- I am ____ hungry. *now needs to be an adverb*
- I am ____ hungry, and I can eat half a pig.



Bidirectional model!

v different from forecasting; in text you do want what is in the future

Sentiment analysis with RNN

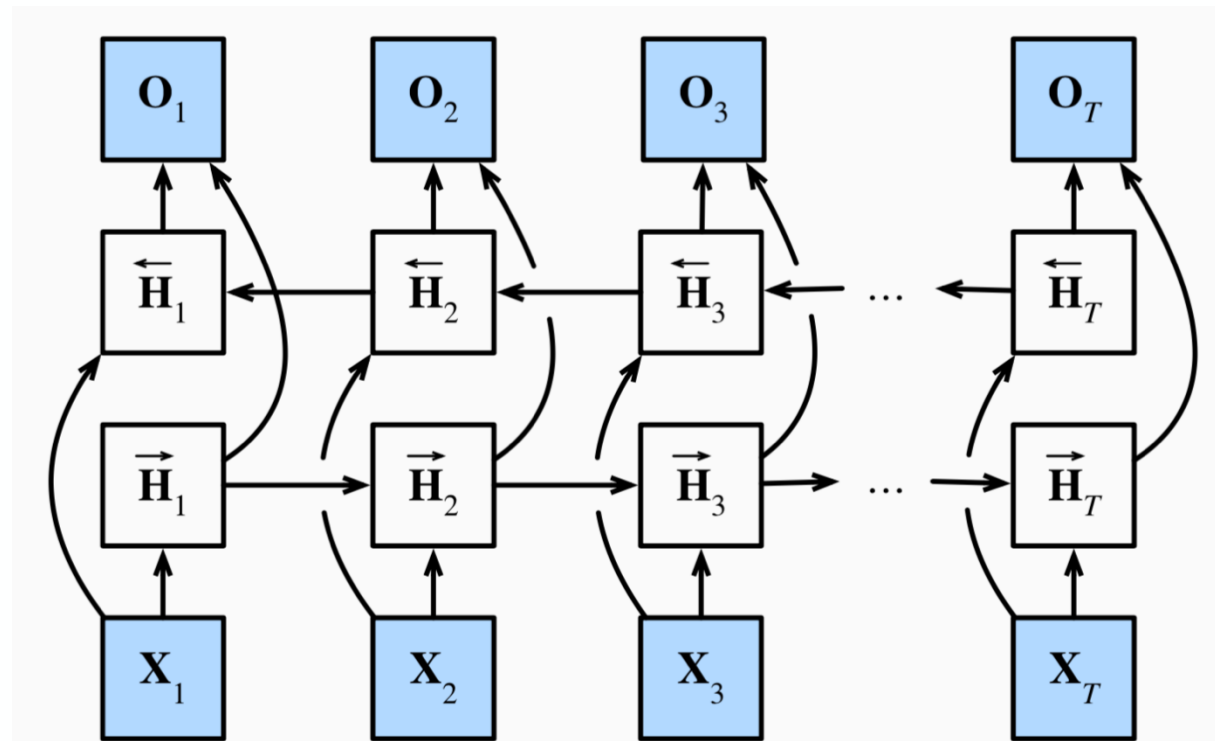


Fig. 10.4.1 Architecture of a bidirectional RNN.

Bidirectional RNNs

We concatenate forward and backward hidden states $\vec{H}_t$ and $\overleftarrow{H}_t$ to obtain the hidden state

$$H_t \in \mathbb{R}^{n \times 2h}$$

(where n is the batch size and h is the number of hidden units in each direction).

H_t is then fed into the output layer.

Sentiment analysis with RNN

Training the model

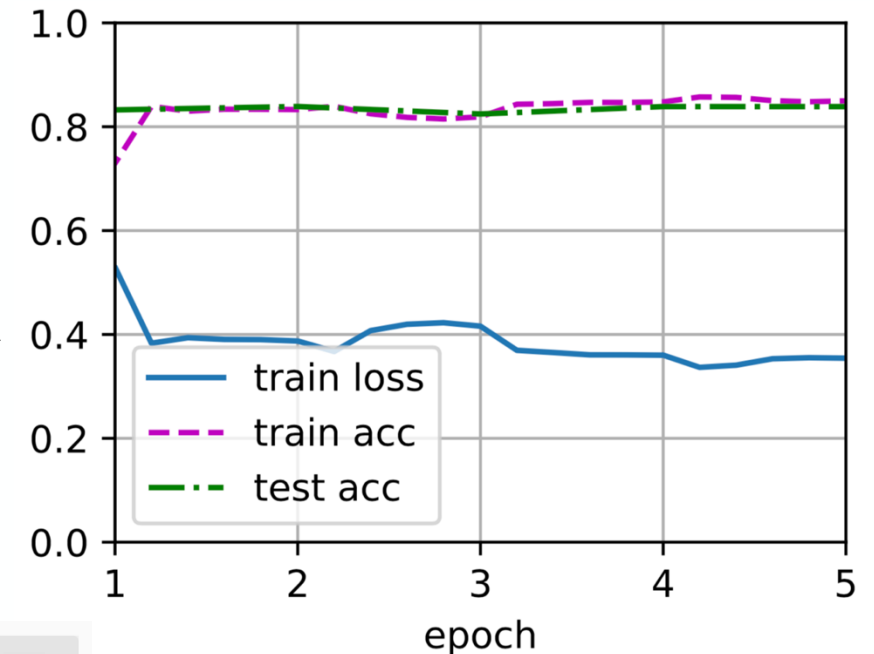
We can then use this to train the model as we have done before

- Example from Zhang et al.: Large Movie Review Dataset (<https://ai.stanford.edu/%7Eamaas/data/sentiment/>)
- 25,000 highly polar movie reviews for training, and 25,000 for testing
- label: 1 review: "Henry Hathaway was daring, as well as enthusiastic, for his..."
- Applying the model:

```
predict_sentiment(net, vocab, 'this movie is so bad')
```

```
'negative'
```

Image: Zhang et al.



Regularization

Regularization

Regularization against overfitting

Recall from Session 3

- **Problem:** Training error very low, poor validation performance.
- Regularization techniques help to reduce the model complexity
 - Weight sharing (which greatly reduces the number of parameters)
 - Weight decay
 - Dropout
- They can also help to make models computationally efficient

Weight sharing

Reducing the number of parameters with new architecture

CNNs: Convolutional layers share weights by applying the same filter to different parts of the image.

RNNs: Weights are shared between each step. We compute the hidden state as:

$$H_t = g(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

The weights W_{xh} , W_{hh} and b_h are shared between each step.

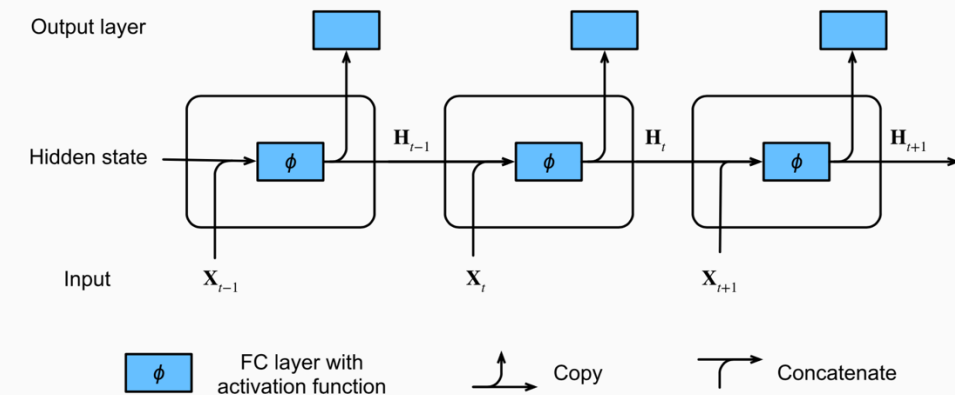
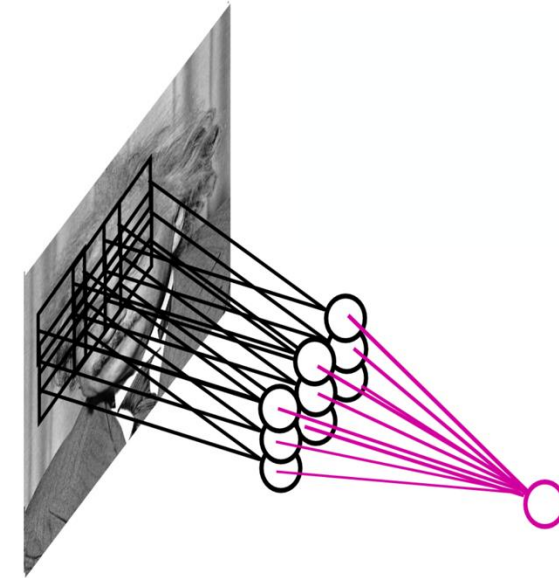


Fig. 9.4.1 An RNN with a hidden state.

Weight decay

Adding a penalty term

- Recall LASSO and ridge regression
- Similarly, in deep learning we can add a penalty term to the loss function that shrinks some weights
- Weight decay is most widely used technique for regularization and uses L_2 regularization
- L_2 norm puts a penalty on the weights
- We can rewrite the loss function as

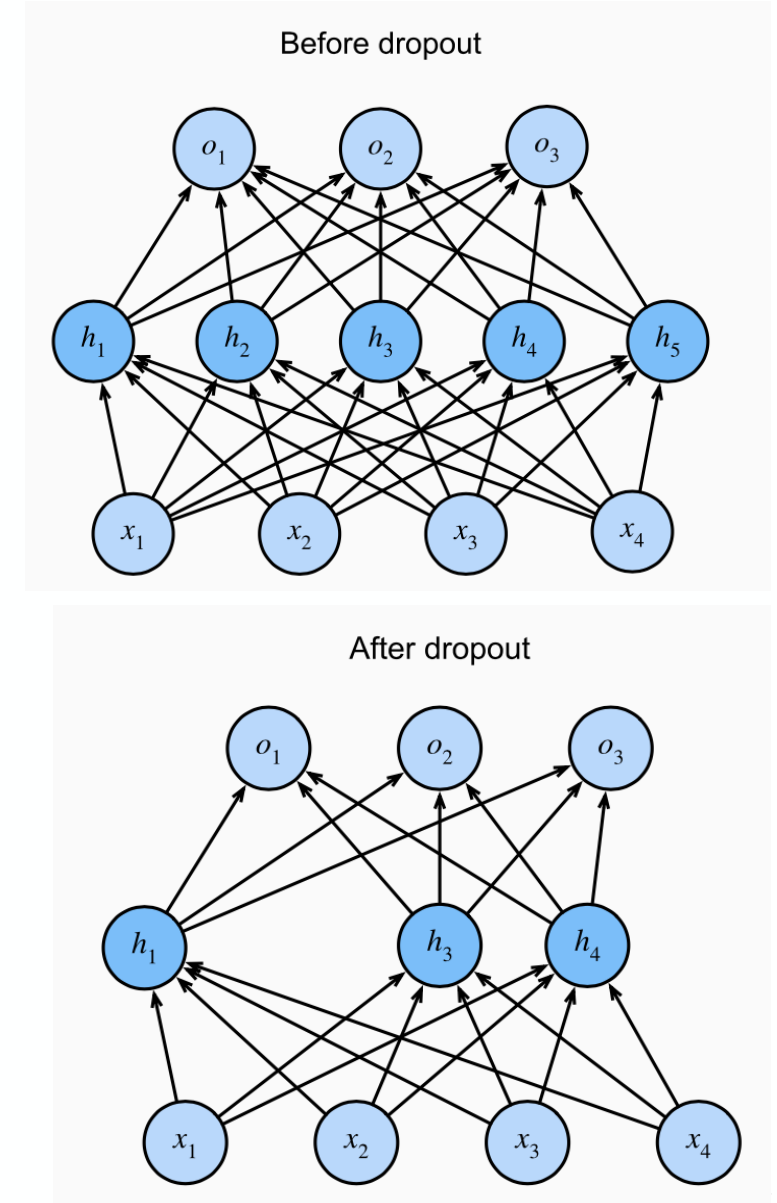
$$L_{new} = L_{original}(W) + \lambda \|W\|_2^2$$

- New objective: minimizing the sum of the prediction loss **and** the penalty term
- The weight update in gradient descent carries a term that “decays” the weight (shrinks towards zero)

Dropout

Injecting noise

- Injecting noise while computing each internal layer during forward propagation to reduce overfitting
- Dropout: We literally drop out some neurons during training.
- Procedure:
 - Throughout training in each epoch: zeroing out some fraction of the nodes in each layer before calculating the subsequent layer.
 - Debias each layer by normalizing by the fraction of nodes that were retained (not dropped out)
 - Disable dropout at test time.



- Text and public policy
- Word embeddings
- Word2vec with focus on skip-gram model
- Continuous bag of words, GloVe
- Sentiment analysis
- Regularization

Hertie School
Friedrichstraße 180
10117 Berlin, Germany
T +49 (0)30 259219-0
F +49 (0)30 259219-11
info@hertie-school.org
www.hertie-school.org